Doloso_Felixander_ACT1

October 22, 2026

0.1 Import required libraries

[ ]:
```
import tensorflow as tf
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.compose import make_column_transformer
from sklearn.preprocessing import MinMaxScaler, OneHotEncoder
from sklearn.model_selection import train_test_split
from tensorflow.keras import layers, models, callbacks
```

0.2 Read in the insurance dataset

[ ]:
```
insurance = pd.read_csv(r"C:
↪\Users\Admin\Desktop\Machine_Learning\Activity_2\insurance.csv")
```

0.3 Check out the insurance dataset

[ ]:
```
insurance.head()
```

0.4 Turn all categories into numbers

[ ]:
```
ct = make_column_transformer(
(MinMaxScaler(), ["age", "bmi", "children"]),
(OneHotEncoder(handle_unknown="ignore"), ["sex", "smoker", "region"])
)
X = insurance.drop("charges", axis=1)
y = insurance["charges"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,␣
↪random_state=42)
ct.fit(X_train)
X_train_normal = ct.transform(X_train)
X_test_normal = ct.transform(X_test)
```

[ ]:
```
X_train.loc(0)
```

[ ]:
```
X_train_normal[0]
```

[ ]:
```
X_train_normal.shape, X_test_normal.shape
```

0.5 Normalize the data and build the model

[8]:
```
tf.random.set_seed(42)

insurance_model_1 = tf.keras.Sequential([
  tf.keras.layers.Input(shape=(X_train_normal.shape[1],)),
  tf.keras.layers.Dense(128, activation="relu"),
  tf.keras.layers.Dense(64, activation="relu"),
  tf.keras.layers.Dense(32, activation="relu"),
  tf.keras.layers.Dense(1)
])

insurance_model_1.compile(loss=tf.keras.losses.mae,
  optimizer=tf.keras.optimizers.Adam(learning_rate=0.01),
  metrics=['mae'])

history_1 = insurance_model_1.fit(X_train_normal, y_train, epochs=1000,␣
 ↪verbose=0)

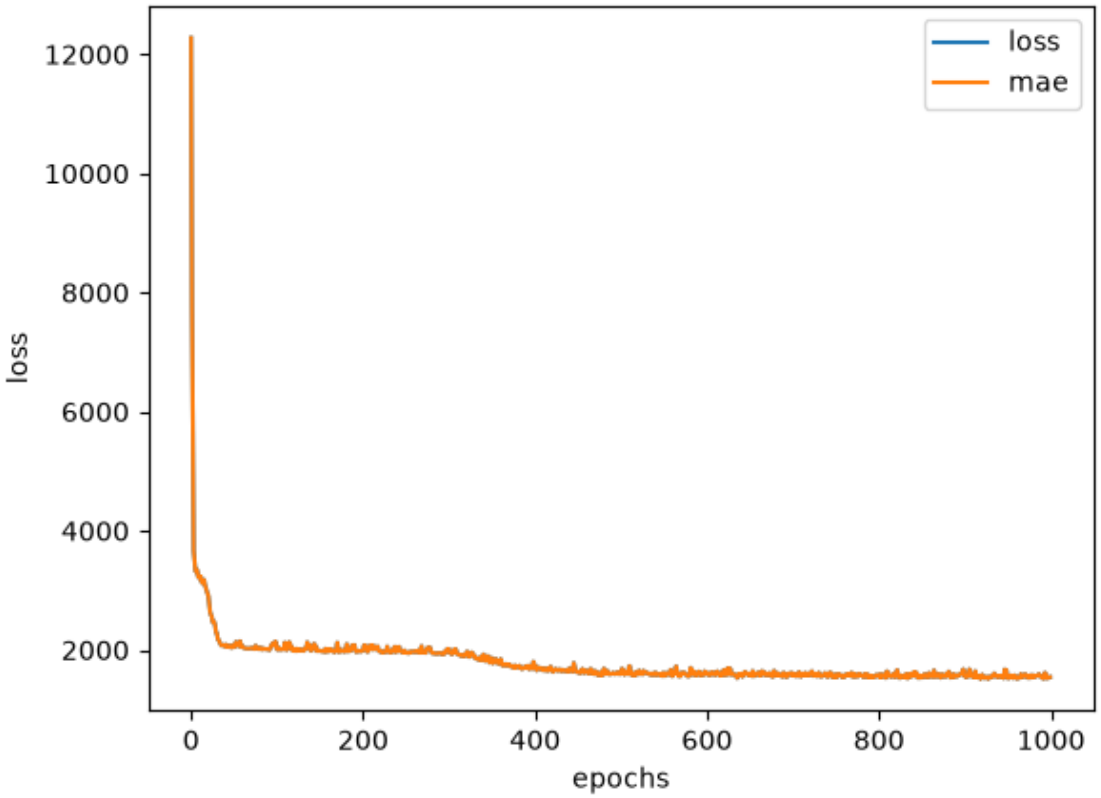
insurance_model_1_loss, insurance_model_1_mae = insurance_model_1.
 ↪evaluate(X_test_normal, y_test)
insurance_model_1_loss, insurance_model_1_mae
```

```
9/9             1s 17ms/step - loss:
1434.9243 - mae: 1434.9243
```

[8]:
```
(1434.92431640625, 1434.92431640625)
```

[9]:
```
pd.DataFrame(history_1.history).plot()
plt.ylabel("loss")
plt.xlabel("epochs")
```

[9]:
```
Text(0.5, 0, 'epochs')
```



0.6 Model No.2

[10]:
```
tf.random.set_seed(42)

insurance_model_2 = tf.keras.Sequential([
  tf.keras.layers.Input(shape=(X_train_normal.shape[1],)),
  tf.keras.layers.Dense(128, activation="relu"),
  tf.keras.layers.Dense(64, activation="relu"),
  tf.keras.layers.Dense(32, activation="relu"),
  tf.keras.layers.Dense(16, activation="relu"),
  tf.keras.layers.Dense(1)
])

insurance_model_2.compile(loss=tf.keras.losses.mae,
  optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
  metrics=['mae'])

history_2 = insurance_model_2.fit(X_train_normal, y_train, epochs=500,␣
 ↪verbose=0)
insurance_model_2_loss, insurance_model_2_mae = insurance_model_2.
 ↪evaluate(X_test_normal, y_test)
```

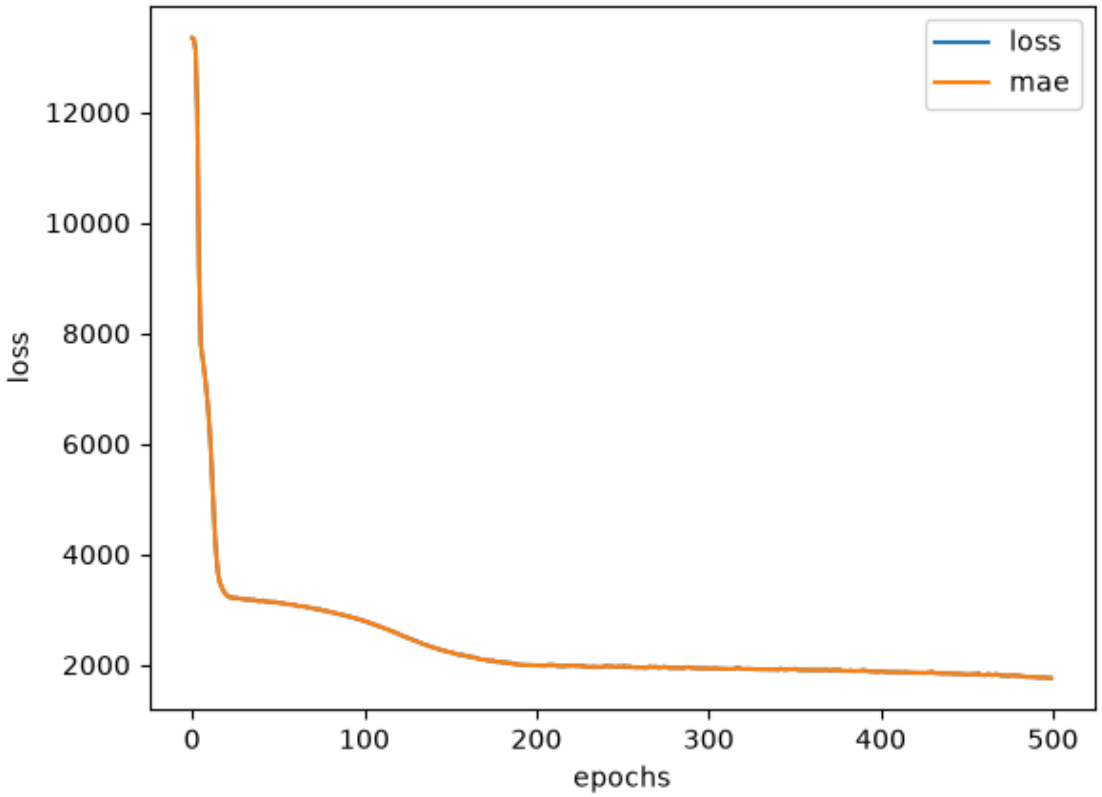
```
insurance_model_2_loss, insurance_model_2_mae
```

```
9/9             1s 9ms/step - loss:
1555.6107 - mae: 1555.6107
```

[10]:
```
(1555.6107177734375, 1555.6107177734375)
```

[11]:
```
pd.DataFrame(history_2.history).plot()
plt.ylabel("loss")
plt.xlabel("epochs")
```

[11]:
```
Text(0.5, 0, 'epochs')
```



0.7 Model No.3

[12]:
```
y_scaler = MinMaxScaler()
y_train_scaled = y_scaler.fit_transform(y_train.values.reshape(-1, 1))
y_test_scaled = y_scaler.transform(y_test.values.reshape(-1, 1))

tf.random.set_seed(42)

insurance_model_3 = tf.keras.Sequential([
```

```
  tf.keras.layers.Input(shape=(X_train_normal.shape[1],)),
  tf.keras.layers.Dense(128, activation="relu"),
  tf.keras.layers.Dense(64, activation="relu"),
  tf.keras.layers.Dense(32, activation="relu"),
  tf.keras.layers.Dense(16, activation="relu"),
  tf.keras.layers.Dense(1)
])

insurance_model_3.compile(loss=tf.keras.losses.mae,
  optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
  metrics=['mae'])

early_stop = callbacks.EarlyStopping(
    monitor="val_loss",
    patience=15,
    restore_best_weights=True)

history_3 = insurance_model_3.fit(X_train_normal, y_train_scaled,
                        validation_split=0.2,
                        batch_size=32,
                        callbacks=[early_stop],
                        epochs=500,
                        verbose=0)

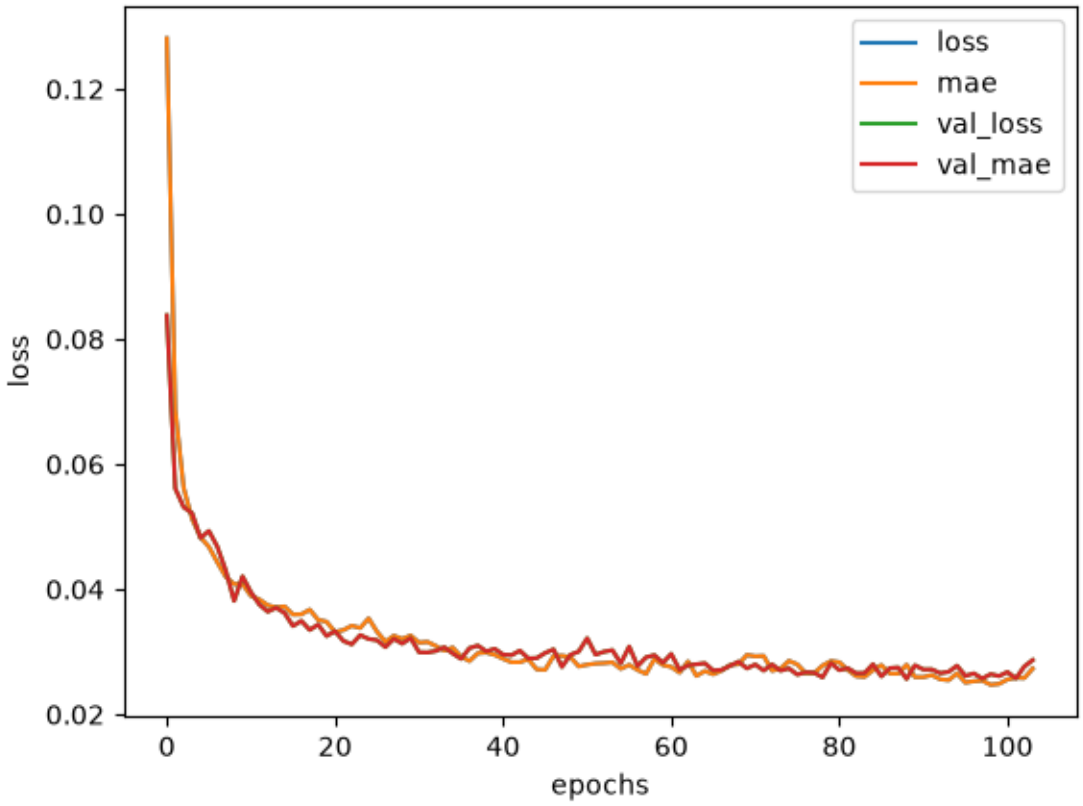
insurance_model_3_loss, insurance_model_3_mae = insurance_model_3.
 ↪evaluate(X_test_normal, y_test_scaled)
insurance_model_3_loss, insurance_model_3_mae
```

```
9/9             0s 14ms/step - loss:
0.0264 - mae: 0.0264
```

[12]:
```
(0.026441778987646103, 0.026441778987646103)
```

[13]:
```
pd.DataFrame(history_3.history).plot()
plt.ylabel("loss")
plt.xlabel("epochs")
```

[13]:
```
Text(0.5, 0, 'epochs')
```


loss
mae
val_loss
val_mae
0.12
0.10
0.08
0.06
0.04
0.02
loss
0
20
40
60
80
100
epochs